

Governance Overview

Purpose

The design system should be managed as a shared platform product. Teams should be able to propose evidence-backed improvements, contribute implementation work, and help maintain the system, while maintainers control what becomes an approved shared capability.

This governance layer answers a different question than the runtime architecture:

- Runtime architecture: how tokens, wrappers, shell, remotes, and validation technically work.
- Governance: how teams decide what belongs in the shared system, how evidence is reviewed, and how approved guidance is published.

Core Principle

Make contribution easy. Make evidence expected. Make promotion controlled.

The design system should be governed as a product. Governance defines how changes are proposed, reviewed, released, communicated, measured, and revisited.

That means the model is neither fully centralized nor fully open-ended:

Model	Risk
Too centralized	Teams wait, create workarounds, or cannot ship.
Too decentralized	Apps create duplicate components, variants, and themes.
InnerSource governance	Teams contribute; maintainers approve shared contracts.

Zeroheight Recommendation

Zeroheight should be the governed front door for approved design-system guidance, lifecycle status, ownership, migration information, and evidence. It should record and communicate decisions made by the responsible design, engineering, platform, and accessibility owners.

It should not create runtime tokens, compile packages, configure `remoteEntry` URLs, or automatically approve promotion.

Use this compact operating model:

```
People decide
Repositories implement
Tests provide evidence
Zeroheight publishes the governed result
```

Governance Flow

```
request or opportunity
-> evidence-backed contribution proposal
-> triage and prioritization
-> design discovery
-> design approval
-> development contribution
-> technical review
-> Storybook and runtime validation
-> promotion decision
-> Zeroheight publication
-> enterprise adoption
```

The important first step is intake. A design-system change should explain the problem, impacted users, reuse evidence, business value, and why the solution belongs in the shared system instead of one application.

The intake model should be treated as a contribution proposal, not as a ticket queue where product teams ask a central group to build everything. A proposed change is a product investment decision for the shared platform.

Contribution Proposal

Before design or code starts, the proposing team should provide enough context for maintainers to decide whether the work belongs in the design system.

Required information:

- problem statement;
- user impact;
- business impact;
- evidence or data;
- existing solutions reviewed;
- proposed scope;
- requesting team;
- expected consumers;
- proposed owner.

Useful evidence can come from user research, customer feedback, support tickets, duplicate implementations, accessibility defects, repeated bugs, compliance needs, or enterprise standardization requirements.

Example:

Four applications implement separate status indicators, and two have known accessibility defects because status is communicated by color alone.

That proposal is stronger than:

We need a status chip component.

Decision Questions

Before a new shared component, token, or pattern is promoted, governance should answer:

- What problem are we solving?
- Who experiences the problem?
- How often does it occur?
- What evidence shows this should be shared?
- Does an existing pattern already solve it?
- Is this a component, pattern, token, or product feature?
- Does it improve accessibility, consistency, maintainability, or reuse?
- Which team owns it after publication?
- What evidence proves it works?

Do Not Promote Gate

Not every useful UI belongs in the design system.

Keep work local when it is:

- single-application only;
- tightly coupled to one domain workflow;
- unsupported by reuse evidence;
- better built as application composition over existing primitives.

Promote work when it is reusable across domains, reduces duplication, improves accessibility or consistency, and has enough evidence to support other teams.

InnerSource Model

Anyone can propose:

- ideas;
- component improvements;
- Storybook examples;
- documentation;
- pull requests;
- migration paths.

Only authorized maintainers and named decision owners may approve changes to protected shared contracts, including:

- token changes;
- breaking API changes;
- core component APIs;
- accessibility standards;
- PrimeNG provider mapping;
- PrimeNG preset and theme changes;
- runtime integration behavior.

This lets product teams contribute without allowing uncontrolled forks.

Contributor Progression Model

Contributors can build trust and influence over time through consistent participation:

Level	What It Means	Permissions	Path to Next Level
User	Uses the design system	Read access to repos and Zeroheight	Become a Contributor
Contributor	Submits PRs and proposals	Create branches, submit PRs	Demonstrated quality across 3+ contributions
Repeat Contributor	Regular participator with proven judgment	PR review within domain, self-merge after approval	Nominated by maintainer after 6+ months consistent quality
Trusted Committer	Recognized reviewer for specific areas (e.g., form components, ally)	Review and merge within agreed scope, help mentor new contributors	Appointed by Design System Council
Maintainer	Full repository ownership	All permissions, releases, breaking changes	Proposed by council, requires consensus

This progression lets product-team developers become recognized reviewers for areas of expertise without joining the central design-system team.

Governance Levels

“InnerSource” means different things in different contexts. This design system operates at the **Contributions Welcome** level:

Level	Meaning	Your System
Visible	Anyone can read, but no participation	Not this
Feedback Welcome	Anyone can file issues; maintainers decide on changes	Not this
Contributions Welcome	Anyone can propose and implement; maintainers control approval	âœ… This level
Shared Write Access	Trusted contributors can review and merge within scope	Future progression for Trusted Committers
Shared Ownership	Multiple teams jointly control direction	Not planned

This clarity prevents confusion: contributors may not have merge rights, but they are genuinely invited to propose work.

Repository Readiness Standard

Before a repository accepts InnerSource contributions, it must provide self-service onboarding:

Required files and clarity:

- README.md âœ” Purpose, key links, getting started
- CONTRIBUTING.md âœ” How to propose work, coding standards, review expectations, escalation path
- COMMUNICATION.md âœ” Where discussions happen (GitHub issues, Slack, email), response times, office hours
- Named maintainers âœ” Who reviews PRs, makes decisions, and can be contacted

- Local setup â€” Build and test commands that work
- Coding standards â€” Linting, formatting, testing requirements
- Review turnaround â€” Maximum time before first feedback
- Escalation path â€” What happens if you're blocked

This turns onboarding into a self-service process instead of requiring private conversations with maintainers.

Contribution Service Contract

Contributors and maintainers make implicit promises to each other. Make them explicit:

Maintainers promise:

- New proposals acknowledged within 3 business days
- First PR review started within 5 business days
- A maintainer assigned before substantial implementation begins
- Clear feedback on blockers or needed changes
- Escalation path if review is delayed

Contributors promise:

- Follow the contribution template and provide required context
- Remain available for code review feedback
- Support the code through an agreed stabilization period after merge (typically 2 weeks)
- Address defects discovered during that period
- Work with maintainers on any adoption issues

This service contract reduces fear on both sides: maintainers know contributors won't abandon code, and contributors know reviews won't take months.

Evidence Gates

Promotion should depend on evidence, not just completion of code.

Gate	Evidence
Contribution proposal	Problem, impact, reuse case, requesting team, expected consumers, proposed owner.
Triage	Reuse, extend, create, keep local, or research decision.
Design approval	Figma proposal, states, accessibility, token requirements.
Technical review	API, token usage, PrimeNG mapping, maintainability, security.
Storybook contract	Supported states, variants, content, theme behavior.
Runtime validation	Shell, remote, Playwright, accessibility, overlay proof when needed.
Publication	Zeroheight page with owner, status, guidance, evidence, change notes.

Zeroheight Role

Zeroheight is the visible governance and adoption layer.

A useful source-of-truth split is:

Tool	Source Of Truth For
Figma	Design assets, anatomy, interaction intent, and design exploration.
Source repository	Component implementation, token build, tests, and release artifacts.
Storybook	Live implementation examples, states, and component behavior.
Zeroheight	Published guidance, approved usage, lifecycle status, owners, decisions, and evidence links.

It should show:

- what teams should use;
- when to use it;
- approved variants;
- lifecycle status;
- owner;
- accessibility guidance;
- links to Figma, Storybook, source, and validation evidence;
- migration and deprecation guidance;
- change history.

Zeroheight can be the human handoff bridge between Figma and engineering. It records decisions and links evidence; people approve, repositories implement, and tests prove behavior.

Proposed future policy: once the organization approves Zeroheight as the governance channel, guidance that has not been published there should be treated as experimental or not yet approved for broad product-team consumption. Until then, agreed repository documentation should serve as the interim governance record.

Token Governance Path

Token changes need their own governance path because they affect registry wrappers, direct legacy PrimeNG usage, overlays, shell-mounted remotes, and application theming.

token proposal

- > design intent and source-token review
- > semantic mapping approval
- > source-controlled implementation
- > generated CSS and PrimeNG adapter
- > automated validation
- > versioned release
- > Zeroheight token guidance and change notice

Token review should answer:

- Is the source value approved?
- Is the semantic name provider-independent?
- Is the PrimeNG mapping internal to the preset or wrapper layer?
- Are light and dark modes covered?
- Is the change breaking?
- Which teams, components, or remotes are affected?

Lifecycle

Status	Meaning
Idea	Identified need that has not been accepted for discovery.
Proposed	Accepted for discovery and triage, but not yet implemented.
Experimental	Implementation under development; API and guidance may change.
Candidate	Evidence complete enough for promotion review.
Active	Approved, versioned, tested, documented, and supported.
Deprecated	Still available temporarily with migration guidance.
Retired	No longer supported or distributed.

Lifecycle status should be visible in Zeroheight and backed by registry metadata or source-controlled documentation. See [Component Promotion](#) for promotion evidence and decision rules.

Proposed Governance Structure

The roles, approval bodies, metrics, and operating cadence below are proposed starting points. Existing enterprise teams and decision authorities should be mapped onto them before adoption.

Group	Responsibilities
Design system council	Strategy, roadmap, governance decisions, conflict resolution.
UX design team	Visual language, interaction patterns, design tokens, documentation.
Engineering team	Component implementation, Storybook, testing, releases.
Product teams	Proposals, adoption feedback, domain evidence, gap reporting.
Accessibility team	Compliance review, keyboard, focus, screen reader, contrast standards.
Platform team	Shell, remotes, runtime integration, provider setup.
Executive sponsor	Funding, organizational alignment, adoption expectations.

Ownership Split

UX owns:

- visual intent;
- interaction intent;
- content guidance;
- design documentation;
- token intent;
- variants and states.

Engineering owns:

- component library implementation;
- Storybook;
- tests and CI;
- versioning and release mechanics;
- framework implementations;
- performance;
- backwards compatibility;
- framework behavior.

Shared ownership:

- component APIs;
- breaking changes;
- semantic token contracts;
- accessibility requirements and proof;
- PrimeNG preset and theme changes;
- roadmap priorities;
- adoption metrics;
- deprecation plans.

Accessibility ownership is shared: design specifies accessible intent, engineering proves implementation behavior, and accessibility specialists review where available.

Adoption And Rollout

Governance should be introduced as a way to help teams deliver faster with less repeated work, not as another approval barrier. Product teams need representation, support, review expectations, migration help, and a clear exception path.

Start with a small set of high-value components or patterns, run the full proposal-to-publication process, measure whether teams get value, and expand from proven examples. See [Adoption And Team Buy-In](#) and [Rollout Implementation](#) for the detailed model.

Future Operating Model

The following change classification, adoption, metrics, and cadence sections are proposed operating-model details. They should be confirmed against enterprise governance before being treated as policy.

Change Classification

Every change should be classified before release.

Change	Examples	Governance
Patch	Bug fixes, docs corrections, non-breaking test updates.	Design-system team review.
Minor	New variants, new components, additive APIs.	Council or maintainer review.
Major	Breaking API, visual language shift, provider theme change.	Migration guide and broad notice.

Adoption Standards

Product teams should:

- use existing components before requesting new ones;
- avoid local modifications of system components;
- submit enhancement requests instead of creating forks;
- follow published Zeroheight usage guidance;
- request exceptions when local needs conflict with shared standards.

Exceptions should be approved, owned, time-bound where possible, and documented.

Metrics

Governance should track whether the system is working.

Adoption:

- percentage of applications using the design system;
- percentage of UI built from shared components;
- component reuse rate;
- Figma library adoption.

Quality:

- accessibility compliance;
- visual consistency;
- defect rate;
- regression rate.

Delivery:

- time from request to release;
- time to publish new components;
- development effort saved through reuse.

Documentation:

- Zeroheight coverage;
- Zeroheight page views;
- Storybook usage;
- documented owner coverage.

Operating Cadence

Meeting	Frequency	Purpose
Design system working session	Weekly	Review requests, active work, bugs, and releases.
Design and engineering sync	Bi-weekly	Coordinate upcoming components and implementation.

Meeting	Frequency	Purpose
Design system council	Monthly	Governance decisions, roadmap, standards, exceptions.
Design system health review	Quarterly	Metrics, adoption, deprecations, and strategic investments.

Document Map

Document	Role
Developer Journey	What the design-system experience looks like to an external product-team developer.
Contribution Process	Detailed intake, triage, evidence, and promotion process.
InnerSource Contribution Model	Team participation and maintainer approval model.
Component Promotion	Lifecycle statuses and promotion evidence.
Zeroheight Governance Model	Zeroheight page model, ownership, and publication guidance.
Governance Flow Map	End-to-end flow between design, tokens, registry, validation, and docs.
Adoption And Team Buy-In	Participation, support, and adoption measures.
Rollout Implementation	Gradual rollout sequence and risk-based approval.

InnerSource Contribution Model

Purpose

The design system should be governed as a shared platform capability, not as a closed handoff from one central team to every application team.

This model explains how design, product, platform, accessibility, and application teams can contribute improvements without creating unsupported forks or unreviewed variants.

The model starts with evidence. Teams are not only requesting work from a central design-system team; they are proposing improvements to a shared platform capability and helping prove why those improvements should become shared.

Operating Principle

Contribution should be open. Approval should be controlled.

That means:

- product and application teams can propose needs and contribute code;
- design and UX teams can propose patterns, variants, and interaction changes;
- platform teams can contribute runtime integration requirements;
- maintainers control shared APIs, token changes, accessibility rules, and breaking changes.

The goal is to avoid both extremes:

Extreme	Failure Mode
Too centralized	Teams wait, create local workarounds, or cannot ship.
Too decentralized	Every app creates its own button, table, header, form, or theme.

Contribution Flow

Use [Contribution Process](#) for the detailed intake, evidence, triage, validation, and promotion steps.

Request or opportunity

- > evidence-backed contribution proposal
- > triage and prioritization

- > design proposal
- > design review gate
- > development proposal
- > code review and InnerSource gate
- > Storybook contract
- > runtime validation
- > component registry promotion
- > Zeroheight governance publication
- > enterprise adoption

Stage 0: Contribution Proposal

A team identifies a need.

Example:

Loan pipeline needs a new status visualization.

Before creating a new shared asset, answer:

- What problem are we solving?
- Who experiences the problem?
- What evidence shows the problem is reusable?
- Does this already exist?
- Is this domain-specific?
- Will another application need it?
- Is this a component, a pattern, or a feature?
- Does it require new tokens?
- Does it affect accessibility, content, or interaction standards?

Output:

- short evidence-backed component, pattern, token, or preset/theme proposal;
- expected consumers;
- initial owner;
- recommendation to reuse, extend, or create.

Stage 1: Design Proposal

Design creates or updates the Figma artifact.

Required design inputs:

- user problem;
- component anatomy;
- states;
- variants;
- responsive behavior;
- interaction behavior;
- accessibility expectations;
- token requirements;
- content guidance where relevant.

Design review decides whether the proposal is:

Decision	Meaning
Existing pattern	Use or extend what already exists.
New shared component	Candidate for registry contribution.
Composite pattern	Reusable workflow made from multiple pieces.
Product-specific feature	Keep in the application, not the registry.

Reviewers:

- design-system design owner;

- UX leadership or delegated design reviewer;
- accessibility reviewer when behavior or semantics are affected.

Output:

- approved design specification or rejection with rationale.

Stage 2: Development Proposal

Developers create or update the implementation.

Expected delivery:

- Angular implementation;
- PrimeNG wrapper or provider mapping when needed;
- public API in design-system language;
- token usage;
- unit tests where useful;
- Storybook stories;
- accessibility notes;
- migration notes if replacing direct PrimeNG or an older pattern.

Implementation should live in the registry package when it is shared. Product specific workflows should stay in the application and consume lower-level design-system primitives.

Stage 3: Code Review And InnerSource Gate

Contributions are reviewed like shared platform code.

Reviewers:

- component owner;
- design-system maintainer;
- platform engineer when shell, remotes, overlays, or theme behavior are affected;
- accessibility reviewer for keyboard, semantics, contrast, or focus changes;
- security or product reviewer when enterprise standards require it.

Review criteria:

- API consistency;
- accessibility;
- token alignment;
- PrimeNG encapsulation;
- performance;
- maintainability;
- test coverage;
- migration impact;
- versioning and release impact.

Protected areas require maintainer approval:

- token source files and normalization rules;
- core component APIs;
- accessibility rules;
- breaking API changes;
- PrimeNG preset/provider changes;
- shell/runtime integration behavior.

Stage 4: Storybook Contract

Storybook proves isolated behavior before broad adoption.

Required stories should cover applicable states:

- default;
- disabled;
- loading;

- error or validation state;
- empty state;
- long content;
- dark theme;
- responsive layout;
- keyboard behavior;
- accessibility expectations.

When Storybook coverage is sufficient, the component can move to **Candidate**.

Stage 5: Runtime Validation

Runtime validation proves the component works in the delivery model.

Validation surfaces:

- direct subapplication route;
- shell-mounted route;
- /qa route or equivalent integration environment;
- Playwright regression tests;
- accessibility checks.

Runtime validation should specifically cover:

- theme inheritance;
- Web Component boundary behavior;
- PrimeNG provider compatibility;
- overlay behavior when applicable;
- shell and remote compatibility;
- responsive layout inside realistic page chrome.

When runtime validation is sufficient, the component can move from **Candidate** to **Active**.

Stage 6: Registry Promotion

The component registry records the implementation contract.

Registry metadata should include:

- status;
- owner;
- source path;
- package and import path;
- public API;
- token dependencies;
- PrimeNG dependencies hidden behind the wrapper;
- Storybook evidence;
- shell or remote validation evidence;
- accessibility notes;
- migration notes.

Lifecycle:

Idea

- > Proposed
- > Experimental
- > Candidate
- > Active
- > Deprecated

Lifecycle status definitions are maintained in [Component Promotion](#). Use those definitions when recording registry metadata or publishing Zeroheight status.

Stage 7: Zeroheight Publication

Zeroheight publishes the governed adoption guidance.

Zeroheight should show:

- when to use the component;
- when not to use it;
- approved variants;
- lifecycle status;
- owner;
- supported versions;
- accessibility rules;
- Storybook link;
- source link;
- validation evidence;
- migration or deprecation guidance.

Zeroheight is the adoption front door. It does not replace the source repository, token build, component registry, or validation suite.

Roles And Responsibilities

Role	Responsibility
Design system team	Maintains standards and approves shared patterns.
Product teams	Contribute domain needs and business priority.
Developers	Implement and maintain code contributions.
UX team	Owns visual consistency and interaction behavior.
Accessibility team	Validates compliance and accessible interaction.
Platform team	Owns shell, runtime integration, and federation constraints.
Product owners	Confirm business priority and adoption need.
Maintainers	Approve protected areas and shared API changes.

Contribution Rules

Anyone can contribute:

- propose improvements;
- submit pull requests;
- add Storybook examples;
- report gaps;
- improve documentation;
- suggest migration paths.

Only authorized maintainers and named decision owners may approve changes to protected shared contracts, including:

- shared token changes;
- core component APIs;
- accessibility rules;
- breaking changes;
- PrimeNG provider mapping;
- runtime integration behavior.

Approval authority may differ by surface, such as design, accessibility, tokens, code, PrimeNG preset/theme mapping, and runtime integration.

Governance Outcome

The InnerSource model should make shared contribution easier without weakening quality gates.

The expected result:

- teams can move without waiting for one central team to build everything;
- reusable improvements flow back into the shared system;
- application-specific features stay in applications;
- approved shared components have clear owners, evidence, and lifecycle status;
- Zeroheight shows teams what is approved and how to adopt it.

Contribution Process

Purpose

A design-system change is a product investment decision, not only a UI request. This process explains how an evidence-backed proposal becomes a shared token, component, pattern, PrimeNG preset/theme change, or documentation update.

The goal is:

- make contribution easy;
- make evidence expected;
- make promotion controlled.

Governance Flow

```
request or opportunity
-> evidence-backed contribution proposal
-> triage and prioritization
-> design discovery
-> design approval gate
-> development contribution
-> technical review gate
-> validation evidence
-> promotion decision
-> Zeroheight publication
```

Contribution Proposal

Before design or code begins, the requesting team should submit a short contribution proposal. This is not a ticket asking a central team to build a component. It is a case for why the shared platform should invest in a reusable capability.

Required information:

- problem statement;
- user impact;
- business impact;
- evidence or data;
- existing solutions reviewed;
- proposed scope;
- requesting team;
- expected consumers;
- proposed owner.

Problem

Describe the user or developer problem.

Example:

```
Loan pipeline teams are implementing status indicators independently, creating
inconsistent accessibility behavior and duplicated logic.
```

Evidence

Not every request needs a large analytics package, but stronger requests include evidence from multiple angles.

User evidence:

- user research;
- customer feedback;
- usability testing;
- support tickets.

Example:

Twelve usability sessions showed users missed status changes because color was the only indicator.

Application evidence:

- number of applications needing the capability;
- existing duplicate implementations;
- maintenance burden;
- inconsistent behavior across products.

Example:

Four applications currently implement separate status indicators.

Technical evidence:

- accessibility gaps;
- performance issues;
- security concerns;
- repeated defects;
- inconsistent keyboard or focus behavior.

Example:

Three applications have different keyboard behavior for the same interaction.

Business evidence:

- regulatory requirement;
- compliance requirement;
- enterprise standardization need;
- product priority.

Example:

WCAG remediation requires consistent focus behavior.

Triage And Prioritization

Design-system maintainers triage the request before design exploration expands.

Questions:

- Is this already solved?
- Is this reusable?
- Is this a pattern, component, token, or product feature?
- Does it reduce duplication?
- Does it improve accessibility?
- Does it support multiple domains?
- Does it belong in the shared registry or inside one application?
- Does it require a token change?
- Does it create a breaking API or migration concern?

Possible outcomes:

Outcome	Meaning
Reuse existing	Existing guidance or component covers the need.
Extend existing	Existing component needs a governed variant or behavior.
New shared asset	Candidate for design-system contribution.
Keep local	Useful, but domain-specific or not reusable enough.
Needs research	Problem is real, but the shared shape is not clear yet.

The outcome should be recorded so teams understand whether the request became shared-system work, stayed local, or needs more evidence.

Do Not Promote Gate

Not every useful UI belongs in the design system.

Keep a request inside the application when it is:

- single-application only;
- a domain-specific workflow;
- unsupported by reuse evidence;
- tightly coupled to one product data model;
- better expressed as application composition over lower-level primitives.

Example local feature:

Loan Pipeline Escalation Risk Analyzer

Likely shared candidates:

- status badge;
- table;
- page header;
- form pattern;
- notification;
- navigation pattern.

Design Discovery

If triage says the request may become shared, design discovery begins.

Participants:

- UX;
- product;
- design-system design owner;
- engineering;
- accessibility where relevant.

Deliverables:

- Figma proposal;
- user flows;
- states;
- variants;
- responsive behavior;
- accessibility requirements;
- token requirements;
- content guidance.

Design Approval Gate

Design approval confirms the proposed shared behavior before implementation.

Approval criteria:

- user need is validated;
- pattern is defined;
- variants and states are explicit;
- accessibility has been reviewed;
- token needs are identified;
- product-specific behavior is separated from reusable behavior.

Output:

- approved design specification;
- decision to create, extend, or keep local;
- named owner for the next stage.

Development Contribution

Development turns the approved design into a governed implementation.

Expected implementation evidence:

- Angular implementation;
- component API;
- PrimeNG wrapper or mapping when needed;
- token usage;
- unit tests where useful;
- Storybook stories;
- accessibility notes;
- migration guidance where relevant.

Technical Review Gate

Technical review validates the contribution as shared platform code.

Review criteria:

- API consistency;
- performance;
- accessibility;
- maintainability;
- security;
- token alignment;
- PrimeNG encapsulation;
- release and migration impact.

Required reviewers depend on the affected surface:

Surface	Reviewer
Core component API	Component owner or design-system maintainer.
Tokens	Token pipeline owner.
PrimeNG preset or theme mapping	Token pipeline owner and platform owner.
Accessibility behavior	Accessibility reviewer.
Shell or remote integration	Platform owner.
Breaking changes	Maintainers and affected product owners.

Validation Evidence

Validation proves the contribution works before promotion.

Required evidence should include:

- Storybook examples;
- `/qa` or shell-mounted route when runtime context matters;
- Playwright regression checks for high-risk behavior;
- accessibility checks;
- theme checks;
- source and package links.

High-risk areas require stronger validation:

- overlays;
- dialogs;
- toasts;
- tables;
- form controls;
- navigation;
- focus management.

Promotion Decision

Promotion changes the lifecycle status.

Idea

- > Proposed
- > Experimental
- > Candidate
- > Active
- > Deprecated

Lifecycle status definitions are maintained in [Component Promotion](#). Use those definitions for components, patterns, token groups, and preset/theme changes unless a more specific status model is approved.

Promotion should be evidence-backed, not automatic. A successful build is not the same as an approved shared component.

Zeroheight Publication

Zeroheight publishes the governed adoption guidance after the required evidence exists.

Zeroheight should show:

- usage guidance;
- approved variants;
- lifecycle status;
- owner;
- implementation link;
- Storybook link;
- validation evidence;
- accessibility notes;
- change history;
- migration guidance.

Zeroheight becomes the institutional memory for adoption decisions. It does not replace source control, tests, the token build, or the component registry.

InnerSource Rule

Anyone can propose:

- ideas;
- component improvements;
- documentation improvements;
- Storybook examples;
- code changes;
- migration paths.

Only authorized maintainers and named decision owners may approve changes to protected shared contracts, including:

- token changes;
- breaking API changes;
- core components;
- accessibility standards;
- PrimeNG provider mapping;
- runtime integration behavior.

Developer Journey: External Team Perspective

Purpose

Governance should feel like a clear paved road to external product-team developers, not a committee process or approval bottleneck.

This document describes what the design-system governance model looks like to a developer from an external team trying to build a new view using approved patterns. It is the practical experience of the InnerSource and governance model from the consumer's perspective.

The Developer's Experience: Eight Steps

1. Start in Zeroheight

The developer goes to Zeroheight to answer:

- Which page-layout pattern is approved?
- Which header, card, form, table, status, and navigation patterns should I use?
- Which variants are allowed?
- What accessibility rules apply?
- What package and version should I install?
- Where are the Storybook examples?
- Is the pattern Active, Candidate, or Experimental?

For a new Rates Sheet view, they might find:

```
âœ… Page layout: Active
âœ… Page header: Active
âœ… Filter panel: Active
âœ… Data table pattern: Active
âœ… Status chip: Active
âœ… Empty state: Active
```

Zeroheight is the starting point for discovery, but not the code source. The Zeroheight page should link directly to:

- Storybook story with working examples
- Figma design reference
- Source repository and current version
- Known issues or deprecation status
- Accessibility guidance
- Link to contribute or request changes

2. Install the Approved Packages

The developer installs the design-system artifacts from the repository:

```
pnpm add @public-sector/tokens @public-sector/primeng-preset @public-sector/ui-patterns
```

The application registers the shared PrimeNG configuration:

```
import { providePrimeNG } from 'primeng/config';
import { publicSectorPrimePreset } from '@public-sector/primeng-preset';

export const appConfig: ApplicationConfig = {
  providers: [
    providePrimeNG({
      theme: {
        preset: publicSectorPrimePreset,
      },
    }),
  ],
};
```

The application also loads the approved token CSS:

```
@use '@public-sector/tokens/css';
```

At this point, the developer has:

- âœ… Approved component wrappers
- âœ… Shared token CSS variables
- âœ… PrimeNG preset configured
- âœ… Accessibility defaults enabled

3. Copy the Nearest Storybook Example

The developer should not invent the view from scratch.

They navigate to the Storybook story linked from Zeroheight and copy a proven composition as a starting template:

```
<public-page-layout>
  <public-page-header
    [title]='Rates Sheet'
    [description]='Review current rates and pricing options'
  />

  <public-filter-panel
    [fields]='filters()'
    (submitted)='onFiltersApplied($event)'
  />

  <public-data-table
    [rows]='rates()'
    [columns]='columns()'
    [loading]='loading()'
    emptyMessage='No rates match the selected filters'
  />
</public-page-layout>
```

The Storybook story shows:

- Supported component states (loading, empty, error, success)
- Responsive behavior on mobile/tablet/desktop
- Keyboard navigation working end-to-end
- Accessibility labels and ARIA attributes
- Theme switching (light/dark mode)
- Example data shapes

4. Own Domain Content, Not Redesigns

The distinction of responsibility should be crystal clear:

Product Team Owns:

- Page title and business copy
- Route and workflow definition
- API calls and data fetching
- Domain models and business logic
- Permissions and authorization
- Loading and error states
- Domain-specific actions and validations
- Interaction sequencing

Design System Owns:

- Spacing and layout rhythm
- Typography hierarchy
- Component states and variants
- Keyboard behavior and shortcuts
- Focus treatment and indicators
- Responsive grid and breakpoints
- Approved color and contrast
- Accessibility rules and compliance
- PrimeNG preset and theming

The operating principle:

Design system supplies the structure, rules, and guardrails.

Product team supplies the business meaning, data, and behavior.

This separation keeps the view consistent without forcing it into a generic mold.

5. Validate Locally

Before opening a contribution or product PR, the developer runs prescribed checks:

```
# Lint code
pnpm lint

# Run unit tests
pnpm test

# Validate components in Storybook
pnpm storybook:test

# Run E2E tests
pnpm e2e
```

For a shell-mounted subapplication, they also validate the view through the integration surface:

```
# Direct subapp
http://localhost:4201/rates-sheet

# Shell-mounted (if configured)
http://localhost:4200/application/rates-sheet

# Run shell and subapp E2E together
pnpm e2e:federation
```

Before shipping, they confirm:

- âœ… Tokens resolve correctly (CSS variables present)
- âœ… PrimeNG overlays are themed (dropdowns, modals, tooltips)
- âœ… Keyboard navigation works (Tab, Enter, Arrow keys)
- âœ… Responsive layout works (mobile, tablet, desktop)
- âœ… Remote loading does not break the view (if shell-mounted)
- âœ… No local CSS overrides conflict with approved patterns
- âœ… No console errors during normal workflows

6. Decide Whether Anything Is Missing

Most views should be built entirely from approved components and patterns.

When the developer encounters a gap, there are three paths:

Path A: Compose from Existing Primitives The existing components can be combined to solve the problem locally:

Page header + card + form + table + empty state

No design-system change required. The product team owns the composition.

Example: A custom “Loan Dashboard” that uses page header, cards, tables, and status chips. This is product-team composition, not a new shared component.

Path B: Domain-Specific Feature Keep the feature in the product application using shared primitives:

Capital Markets Risk Assessment Panel
(uses table + form + status chip + accordion)

It may consume and compose shared primitives without becoming a shared component itself.

Example: A loan-underwriting-specific feature that is unlikely to be reused across domains. It uses approved components but is not promoted as shared.

Path C: Genuine Reusable Gap Submit a contribution proposal to the design system:

Problem:

Four applications need the same table bulk-action pattern.
Current implementations are inconsistent.

Two have keyboard-navigation defects.
Developers keep rebuilding this locally.

Evidence:

- Three separate implementations in Rates Sheet, Pipeline, Committed Loans
- 12+ defects filed on similar patterns
- Design team confirmed this pattern is common
- User research shows bulk actions are frequent workflows

Expected consumers:

- Rates Sheet team
- Loan Pipeline team
- Committed Loans team
- (at least 3 teams to justify shared investment)

Proposed owner:

Capital Markets UI team

Proposed scope:

New ``<public-bulk-actions>`` component wrapping PrimeNG toolbar with icon pattern, keyboard shortcuts, and accessibility validation

Submit this proposal as an issue or through the contribution portal.

7. Receive a Predictable Review

The developer should know in advance:

- **Who reviews?** Named maintainers and decision owners
- **What evidence is required?** Links to Zeroheight and component-promotion.md
- **Expected turnaround?** Published SLA (e.g., 5 business days for Minor, 10 for Major)
- **What is the decision?** One of five outcomes (see below)
- **Exception path?** Link to exception-handling process
- **Result ownership?** Who maintains this after approval?

The response should not simply be “approved” or “rejected.” It should guide the team toward the right implementation path:

Decision	Response	Next Step
Reuse	Use existing component X instead	Link to Zeroheight page, Storybook example, documentation
Extend	Propose an additive variant of existing Y	Minor change approval, extend Storybook, update docs
Create	This should become a shared component	Route to InnerSource contribution process with timeline
Keep Local	This is domain-specific; implement in your application	Provide pattern recommendations but no shared promotion
Research	This needs more evidence; revisit in 6 months	Document the gap for future roadmap discussion

8. Ship Using an Approved Version

Once the view passes application and integration validation, the product team releases it on its own schedule using approved package versions.

The product team does not need to wait for design-system approval unless they are contributing a new shared pattern.

If they contributed a new shared pattern, that pattern follows its own lifecycle:

Experimental (in development)

â†“ (evidence gates pass)

Candidate (ready for review)

â†" (promotion decision approved)
Active (supported, published, discoverable)

Zeroheight is updated only after the shared capability is promoted to Active.

What the Developer Journey Should Feel Like

Happy Path: Using Approved Patterns (90% of work)

Find approved pattern in Zeroheight

â†"

Copy Storybook example

â†"

Add business data and behavior

â†"

Run automated checks (lint, test, e2e)

â†"

Validate in shell integration

â†"

Ship on product team's schedule

Time investment: 2-4 hours **Approval required:** None (already approved) **Governance feels:** Invisible, enabling

Contribution Path: Extending the System (10% of work)

Identify reusable gap

â†"

Gather evidence (existing implementations, team feedback, usage data)

â†"

Submit contribution proposal

â†"

Collaborate with maintainers (design, UX, accessibility, platform)

â†"

Implement with Storybook validation

â†"

Platform validation (shell, remotes, Playwright)

â†"

Promotion decision review

â†"

Zeroheight publication

â†"

Enterprise adoption

Time investment: 2-4 weeks **Approval required:** Design, engineering, accessibility maintainers **Governance feels:** Structured, collaborative, standards-aligned

Key Success Measures

For the Developer

- âœ… I found what I need in Zeroheight in < 5 minutes
- âœ… I can copy a working Storybook example without modification
- âœ… My view works without overriding approved CSS
- âœ… My keyboard navigation works out of the box
- âœ… I did not need to request an exception
- âœ… My PR review focused on my business logic, not design-system compliance

For the Design System

- âœ… > 80% of views are built from approved patterns

- âœ… Contribution proposals are submitted regularly (1-2 per quarter)
 - âœ… Average review turnaround < 5 business days
 - âœ… > 90% of requests resolved through reuse or extension
 - âœ… Defect rate on shared patterns < 0.5 bugs/month
 - âœ… Zero accessibility compliance failures
-

Enabling This Experience

For the developer journey to feel seamless, the design system must provide:

Discovery

- Searchable, current Zeroheight documentation
- Clear guidance on when to use each pattern
- Links to Storybook examples and Figma reference
- Lifecycle status visible upfront (Active vs. Experimental)
- Known issues and deprecation notices

Code

- Copyable Storybook examples with working component compositions
- Stable Angular packages with semantic versioning
- TypeScript types for component APIs
- Automated linting and validation rules
- Example data shapes for common patterns

Support

- Named maintainers for each component or pattern
- Published review SLAs (maximum wait time)
- Contribution templates (for proposals and code)
- Office hours or async support channel
- Documented exception and escalation paths

Validation

- Automated checks (lint, test, accessibility)
 - Storybook snapshot and interaction testing
 - Shell and remote integration testing
 - Theme and token validation
 - Performance regression detection
-

Anti-Patterns That Break the Experience

Broken discovery

- âœ… Zeroheight is out of sync with actual implementation
- âœ… Developers cannot find approved patterns easily
- âœ… Status is unclear (Active? Experimental? Deprecated?)
- âœ… No Storybook link or broken link

Impact: Teams implement their own components instead of reusing

Broken code

- âœ… Storybook examples require modification to work
- âœ… Components have undocumented breaking changes
- âœ… Package versions are unstable or frequently breaking
- âœ… No TypeScript types

Impact: Teams fork components locally instead of upgrading

Broken support

- â Ē No named maintainer or unclear ownership
- â Ē Contribution requests wait > 2 weeks with no feedback
- â Ē Exception process is opaque or undefined
- â Ē No help for teams following the governed path

Impact: Teams bypass the process and create workarounds

Broken validation

- â Ē No automated linting (teams make mistakes uncaught)
- â Ē No shell integration testing (regressions slip through)
- â Ē No accessibility validation (compliance gaps not detected)
- â Ē No performance tracking (regressions accumulate)

Impact: Shared components degrade over time; teams lose confidence

The Core Principle

Following the governed path must be easier than creating a local fork.

When this principle is upheld, governance is mostly invisible during normal development. It becomes visible only when the team needs to change or extend the shared system.

The developer should never think “Ugh, governance is slowing me down.” Instead, they should think “Oh, this example is perfect, I can copy it directly.”

When that happens, the governance model is working.

Component Promotion

The promotion model keeps shared UI from becoming an uncontrolled fork of PrimeNG or a collection of undocumented one-off wrappers.

Promotion begins after intake and triage. A useful UI element does not become a shared component just because it exists in one application; it needs evidence that the capability is reusable, owned, documented, and safe for other teams to adopt.

Lifecycle

Status	Meaning
Idea	Identified need that has not been accepted for discovery.
Proposed	Accepted for discovery and triage, but not yet implemented.
Experimental	Implementation under development; API and guidance may change.
Candidate	Evidence complete enough for promotion review.
Active	Approved, versioned, tested, documented, and supported.
Deprecated	Still available temporarily with migration guidance.
Retired	No longer supported or distributed.

Technical qualifiers such as **Storybook verified**, **shell verified**, or **Accessibility risk** can be tracked separately from lifecycle status.

Promotion Evidence

A component or pattern should not become **Active** until it has enough evidence for other teams to trust it:

- Token alignment
- Approved wrapper usage and PrimeNG provider compatibility
- Supported states and variants
- Storybook stories

- Accessibility checks
- Theme checks
- Direct subapplication validation
- Shell-mounted validation
- Playwright evidence where appropriate
- Zeroheight guidance
- Named owner
- Release notes or migration notes

PrimeNG preset and theme changes should follow the same evidence model because they affect every wrapper, direct legacy PrimeNG usage, overlays, and shell-mounted remotes.

Decision Rules

Search before creating a new shared component. PrimeNG should remain behind a registry wrapper so the component provider can be swapped later. A wrapper may be thin when PrimeNG already satisfies the need with the approved token preset, but application teams should still consume the registry API.

Use a stricter registry wrapper when the system needs to enforce behavior, defaults, Accessibility, telemetry, API consistency, or release control.

Create a composite component or pattern when multiple UI pieces form a repeated workflow that teams should not rebuild independently.

Governance Flow Map

This map shows how design-system decisions, token artifacts, runtime packages, validation evidence, and documentation should move through the platform.

End-To-End Flow

Figma design intent

- > Zeroheight governance and handoff
 - > approved usage, status, owners, evidence links
- > approved DTCG-compatible token export
- > source-controlled token build
 - > tokens.css
 - > design-tokens.json
 - > TypeScript token exports
 - > zeroheight-tokens.json
- > packages/primeng-preset
- > packages/ui-patterns registry wrappers
- > shell, subapps, Storybook, and Playwright
- > promotion lifecycle evidence
- > Zeroheight guidance and governance pages

Zeroheight can sit between design and engineering as the human governance and handoff layer. The repository and generated artifacts remain the implementation source of truth. Zeroheight receives documentation-ready outputs and evidence links; it does not compile runtime tokens or configure applications.

Figma And DTCG Input

Design intent:

- Should happen: Figma captures anatomy, variants, states, accessibility notes, and token intent.
- Should not happen: screenshots alone become runtime values.

Token input:

- Should happen: the token repo consumes a Figma-backed DTCG-compatible JSON artifact.
- Should happen: Zeroheight records the approved decision and links the source artifact, if the team uses it for handoff.
- Should not happen: Zeroheight becomes the token compiler or the only source of production token values.

Validation:

- Should happen: the token build validates naming, aliases, and required metadata.
- Should not happen: ad hoc CSS values bypass token mapping.

Feedback:

- Should happen: implementation gaps go back to design for token or component decisions.
- Should not happen: shipped differences become undocumented overrides.

Token Repo Outputs

Output	Consumer	Purpose
<code>tokens.css</code>	Shell, remotes, Storybook, tests	Runtime CSS custom properties and theme variables.
<code>design-tokens.json</code> TypeScript exports	Tooling, docs, tests PrimeNG preset, tests, tooling	Machine-readable token metadata. Shared values without duplicating token data.
<code>zeroheight-tokens.json</code> PrimeNG preset mapping	Zeroheight export or upload process <code>packages/primeng-preset</code>	Documentation-ready token data. Maps tokens into PrimeNG semantic and component tokens.

The same resolved values should feed runtime CSS, PrimeNG preset values, wrapper behavior, and validation tests.

Runtime Consumption

Boundary	What Flows In	What Flows Out
Shell	Token CSS, approved provider, remote manifest, wrappers	Theme context, mounted remotes, shell e2e evidence
Subapps	Token CSS, approved provider, wrappers	Bootstrapped Web Components and isolated evidence
Registry	CSS variables, token helpers, preset	Wrapper APIs, lifecycle metadata, Storybook examples
PrimeNG preset	Token values and mapping rules	PrimeNG configuration and bridge variables

New and target-state subapps consume `@public-sector/ui-patterns`, not PrimeNG directly. Migrated legacy subapps may keep direct PrimeNG temporarily when the path is allowlisted, owned, and tracked as migration debt.

Zeroheight Flow

Flows Into Zeroheight	Does Not Flow Into Zeroheight
Generated token documentation artifacts such as <code>zeroheight-tokens.json</code>	Runtime token delivery
Component usage guidance	<code>remoteEntry</code> URLs
Lifecycle status and ownership	Shell mounting configuration
Storybook, Playwright, source, and Figma links	Application routing
Migration guidance from direct PrimeNG to wrappers	Component implementation source of truth
Release notes and deprecation notes	PrimeNG preset generation

Zeroheight is a governed communication surface. It should explain approved usage and link evidence, not become a runtime dependency.

Evidence And Promotion Flow

Wrapper or token change
-> rebuild artifacts

- > Storybook isolated component proof
- > shell-mounted or remote Playwright proof
- > accessibility and theme checks
- > registry lifecycle update
- > Zeroheight guidance update

Stage	Evidence
Experimental Candidate	Local implementation or initial Storybook proof. Storybook states, token alignment, and direct remote or shell-mounted proof.
Active	Versioned wrapper, accessibility checks, Playwright evidence, owner, notes, and guidance.
Deprecated	Migration path, replacement guidance, owner, and removal plan.

Storybook proves isolated component behavior. Playwright proves repeatable runtime behavior in shell-mounted and remote contexts. Promotion remains a governance decision backed by evidence, not an automatic build result.

Boundary Owners

Named owners still need to be confirmed in the enterprise repository. Until then, use these role owners for accountability:

Boundary	Role Owner	Owns
Design	Design system design owner	Figma intent, anatomy, variants, states, accessibility notes, token intent.
Platform	Platform/runtime owner	Shell composition, theme context, token delivery, remotes, provider setup.
Token pipeline	Design system engineering owner	DTCG ingestion, mapping rules, artifacts, versioning.
Registry	Component registry owner	Wrapper API, PrimeNG encapsulation, lifecycle, Storybook, migration guidance.
Subapps	Subapp team owner	Wrapper use, local bootstrap, route behavior, legacy PrimeNG allowlist entries.
Quality evidence	QA or platform validation owner	Playwright, accessibility, overlay proof, evidence links.
Zeroheight	Design system governance owner	Guidance, owners, evidence, lifecycle, deprecation notes.

Enterprise validation should replace role owners with named accountable owners or team aliases before the platform documentation is treated as final.

Zeroheight Governance Model

Purpose

Zeroheight should be the visible governance layer for the design system. It answers what teams should use, when they should use it, who owns it, and what evidence proves it is ready.

It should not be the runtime source of tokens, component code, shell routing, or remote loading. Those stay in source-controlled repositories and deployment configuration.

Recommended Flow

Zeroheight can sit between design and engineering as a human governance and handoff layer. That is different from putting Zeroheight in the runtime token pipeline.

Human governance flow:

Figma design intent

- > owner approval and Zeroheight guidance, status, and handoff
- > developer implementation in registry, shell, or subapp
- > Storybook, source, and validation evidence linked back to Zeroheight

Runtime delivery flow:

approved token source

- > source-controlled token build
- > CSS variables, JSON metadata, TypeScript exports, and PrimeNG preset
- > registry, shell, remotes, Storybook, and tests

Design, engineering, platform, and accessibility owners approve. Zeroheight documents and communicates the decision. It should not compile production tokens or serve runtime assets.

The component governance flow is:

Figma design intent

- > component or token candidate
- > registry implementation and token build
- > Storybook isolated evidence
- > shell, remote, Playwright, and accessibility validation
- > Zeroheight approved guidance

Zeroheight is the front door for approved usage. The repository remains the source of implementation truth.

Use this source-of-truth split:

Surface	Authoritative For
Figma	Design assets, anatomy, states, and interaction intent.
Token repository	Runtime token source, generated artifacts, and build validation.
Code repository	Component implementation, APIs, tests, and releases.
Storybook	Live implementation behavior, examples, and states.
Zeroheight	Published guidance, approved usage, lifecycle status, owners, decisions, and evidence links.

What Zeroheight Owns

Zeroheight should own the governed presentation of:

- contribution decisions and rationale where useful;
- component and pattern status;
- usage guidance and anti-patterns;
- approved variants and states;
- accessibility expectations;
- ownership and support contacts;
- migration and deprecation guidance;
- release notes or change summaries;
- links to Figma, Storybook, source, and validation evidence.

What Zeroheight Does Not Own

Zeroheight should not own:

- runtime token delivery;
- token creation or normalization rules;
- `remoteEntry` URLs;
- shell mounting or routing;
- component implementation;
- PrimeNG preset generation;
- automatic promotion decisions.

Automation can publish or sync documentation later, but promotion should remain a governance decision backed by evidence. Zeroheight can also record why a proposal was kept local, deferred, or routed to research. That history helps teams avoid repeating the same debate and makes the design system feel like an actively managed product rather than a static component catalog.

Lifecycle Statuses

Status	Meaning	Use Guidance
Idea	Identified need that has not been accepted for discovery.	Track only; do not use in app work.
Proposed	Accepted for discovery and triage, but not yet implemented.	Do not use in app work.
Experimental	Implementation under development; API and guidance may change.	Use only with owner approval.
Candidate	Evidence complete enough for promotion review.	Pilot in limited scope.
Active	Approved, versioned, tested, documented, and supported.	Preferred path for new work.
Deprecated	Still available temporarily with migration guidance.	Do not use for new work.
Retired	No longer supported or distributed.	Use the replacement.

Technical qualifiers such as **Storybook verified**, **shell verified**, **accessibility risk**, or **legacy migration** can be tracked alongside lifecycle status.

Promotion Gates

A component, token group, or reusable pattern should not move to **Approved** until these questions have clear evidence:

Gate	Required Evidence
Design intent	Figma reference or design approval.
Implementation	Registry source link or token source link.
Token alignment	Uses approved semantic tokens or generated PrimeNG mapping.
Variants and states	Storybook examples for supported states.
Accessibility	Keyboard, semantics, contrast, and disabled/loading behavior checked.
Theme behavior	Light, dark, and approved theme variants checked where applicable.
Integration	Shell or remote validation when runtime context matters.
Automation	Playwright or repeatable validation for high-risk behavior.
Ownership	Named owner or owning team.
Change notes	Release, migration, or deprecation notes where needed.

Overlays, toasts, dialogs, tables, and forms should require integration evidence because they can fail differently in federated remotes.

Component Page Model

Each Zeroheight component page should include:

- purpose;
- lifecycle status;
- owner;
- package and import path;
- selector or public API;
- when to use it;

- when not to use it;
- approved variants and states;
- anatomy;
- behavior;
- accessibility notes;
- responsive behavior;
- content guidance;
- token usage notes;
- examples;
- migration guidance from direct PrimeNG where relevant;
- known limitations;
- Storybook link;
- Figma link;
- source link;
- shell, remote, or Playwright evidence link;
- release history;
- release or deprecation notes.

Example page outline:

Button

Status: Approved
 Owner: Design System Team
 Implementation: @public-sector/ui-patterns / ps-button
 Storybook: [link]
 Source: [link]
 Validation: [Playwright or shell evidence link]

Use for:

- primary actions
- secondary actions
- text actions
- loading actions

Do not use for:

- custom colors
- app-level PrimeNG button styling
- direct p-button usage in new subapps

Token Page Model

Token pages should be generated or reviewed from source-controlled artifacts, not hand-maintained runtime values.

Each token page should explain:

- token purpose;
- semantic usage;
- light and dark values;
- theme variants where applicable;
- deprecated or alias status;
- source artifact or package;
- build or validation evidence;
- known migration notes from older bootstrap artifacts.

The preferred documentation input is a generated artifact such as `zeroheight-tokens.json`, backed by the token build.

Owner Responsibilities

Boundary	Owner	Responsibility
Design	Design owner	Intent, anatomy, variants, states, accessibility notes.
Tokens	Token pipeline owner	Source ingestion, normalization, generated artifacts.
Registry	Registry owner	Wrapper API, lifecycle status, implementation evidence.
Platform	Platform owner	Shell integration, theme context, remote loading evidence.
Subapps	Subapp owner	Adoption, local exceptions, migration debt.
Validation	QA or platform validation owner	Storybook, Playwright, accessibility evidence.
Zeroheight	Governance owner	Published guidance, status, owners, evidence links.

Named accountable owners should replace these role owners when the enterprise repository and operating model are available.

Change Process

1. A designer, platform engineer, or subapp team proposes a token, component, or pattern.
2. The registry owner checks whether an existing pattern already covers it.
3. Design confirms anatomy, variants, states, and content expectations.
4. Engineering implements or updates the token build, wrapper, or pattern.
5. Storybook captures isolated behavior.
6. Playwright, accessibility, shell, or remote validation captures runtime evidence where needed.
7. The governance owner updates the Zeroheight page with status, owner, evidence, and usage guidance.
8. The component or token moves to the next lifecycle status only when the required evidence is linked.

Legacy PrimeNG Policy

New and target-state subapps should use registry wrappers. Migrated legacy subapps may keep direct PrimeNG temporarily when the usage is owned, inventoried, and tracked as migration debt.

Zeroheight should make that distinction visible:

- approved registry path for new work;
- temporary legacy exception where applicable;
- migration guidance from direct PrimeNG to the governed wrapper.

Immediate Next Steps

1. Create the initial Zeroheight information architecture: foundations, tokens, components, patterns, accessibility, release notes, and governance.
2. Define the lifecycle status labels and approval gates in Zeroheight.
3. Publish one complete component page, such as Button, as the reference model.
4. Link that page to Figma, Storybook, source, and validation evidence.
5. Repeat for the first high-risk families: dialog, select, menu, toast, table, and form controls.

Adoption And Team Buy-In

Purpose

Governance should help teams deliver faster with less repeated work. It should not feel like an added approval barrier, a central team blocking delivery, or a documentation burden that developers maintain twice.

Adoption will fail if product teams experience governance as:

- another approval layer;
- a central team blocking delivery;
- duplicate documentation work;

- rules imposed without product-team representation;
- a mandate with no support or migration funding.

The client should build adoption through participation, visible value, and incremental rollout.

Principles

- Product teams help shape the standards they are expected to follow.
- Contribution should be easier than creating and maintaining a local fork.
- Review turnaround expectations should be published.
- Maintainers should provide office hours, examples, migration support, and clear escalation paths.
- Governance should begin with a small set of high-value components and expand based on demonstrated reuse.
- Teams should see how shared components reduce defects, accessibility risk, duplicated effort, and upgrade cost.
- Exceptions should be possible when legitimate domain or delivery constraints exist.

Team Participation

Product teams should not only consume decisions. They should have a formal path to:

- propose improvements;
- contribute implementation or documentation;
- review draft standards;
- challenge decisions with evidence;
- request exceptions;
- influence roadmap priority.

The design-system team should make that path visible in Zeroheight and in the repository contribution docs.

Maintainer Support

Maintainers should provide:

- contribution templates;
- starter code;
- Storybook examples;
- automated checks;
- named reviewers;
- review service-level expectations;
- office hours;
- migration guidance.

The practical goal is that following the governed path is easier than bypassing it.

Measures Of Buy-In

Track whether teams are adopting the model, not only whether governance exists.

- Number of product teams contributing.
- Percentage of requests resolved through reuse or extension.
- Review turnaround time.
- Reduction in duplicate components.
- Adoption of published components and tokens.
- Product-team satisfaction with the contribution process.
- Number and age of approved exceptions.

Anti-Patterns

Governance should be adjusted when:

- teams repeatedly bypass the process;
- reviews take too long for ordinary additive changes;
- documentation diverges from source, Storybook, or tests;
- product teams cannot find owners or escalation paths;
- exceptions are approved but never revisited;

- the system expands without evidence of reuse.

Rollout Implementation

Purpose

The design-system governance model should not begin as an enterprise-wide mandate. The safer rollout is to co-design the model with affected teams, prove it with a real component, measure the value, and expand from working examples.

Practical Sequence

1. Co-design the governance model.
2. Pilot it with a real component or pattern.
3. Show measurable value.
4. Make contribution easier than bypassing.
5. Give product teams representation.
6. Avoid turning governance into a bottleneck.
7. Roll out gradually.

1. Co-Design The Governance Model

Bring together representatives from:

- design system;
- UX;
- platform and shell;
- accessibility;
- product application teams;
- engineering maintainers.

Have those groups agree on decision rights, evidence expectations, approval paths, exception handling, and review responsibilities. Teams support governance more readily when they help design it.

2. Pilot With A Real Component

Use something visible and reusable, such as:

- button;
- status chip;
- page header;
- navigation pattern;
- data-table pattern.

Run the component through the full process:

```
evidence-backed proposal
-> design discovery
-> implementation
-> Storybook proof
-> runtime validation
-> promotion decision
-> Zeroheight publication
-> adoption feedback
```

3. Show Measurable Value

The pilot should demonstrate outcomes such as:

- fewer duplicate implementations;
- consistent accessibility behavior;
- less application-specific styling;
- faster delivery for later adopters;
- simpler PrimeNG upgrades;

- clearer ownership and support.

4. Make Contribution Easier Than Bypassing

Provide:

- contribution templates;
- starter code;
- Storybook examples;
- automated checks;
- named reviewers;
- review service-level expectations;
- office hours;
- migration guidance.

5. Give Product Teams Representation

Product teams need a formal path to:

- propose;
- contribute;
- review;
- challenge;
- request exceptions;
- influence roadmap priority.

6. Use Risk-Based Approval

Not every change needs council-level approval.

Change Type	Review Path
Documentation correction	Lightweight maintainer review.
Additive component variant	Design and component maintainer review.
Token alias or semantic mapping	Token owner and affected component owner review.
Breaking token or API change	Broader governance review and migration plan.
PrimeNG preset or theme change	Token, platform, and registry owner review.

Risk-based review keeps governance from becoming a bottleneck while preserving control over protected shared contracts.

7. Roll Out Gradually

Start with willing product teams and high-value shared patterns. Turn early adopters into advocates before stronger adoption expectations are introduced.

Each rollout wave should identify:

- participating teams;
- target components or patterns;
- migration support needed;
- expected evidence;
- success measures;
- exceptions and owners.

Supporting Documentation

The preceding Zeroheight governance model is the primary recommendation.

The following sections are supporting reference material. They explain the background decisions, flow maps, and promotion rules that back the governance model, but they are not required reading before reviewing the recommendation.